

```
In [27]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.ensemble import RandomForestClassifier

sns.set(style="whitegrid")
pd.set_option("display.max_columns", None)
```

```
In [28]: DATA_PATH = "Order_delivery.csv"
df = pd.read_csv(DATA_PATH)

df.head(10)
```

```
Out[28]:
```

	Order_ID	User_ID	Restaurant_ID	Driver_ID	Item_Name	Quantity	Total_Price	Order_Date
0	1	U3522	358	485	Fried Chicken	3	273.72	16-08-2018
1	2	U9214	316	65	Sandwich	3	365.82	03-08-2018
2	3	U7307	357	309	Koshary	3	401.94	01-08-2018
3	4	U3612	420	32	Sushi	2	221.18	13-08-2018
4	5	U3492	73	364	Koshary	5	355.55	06-08-2018
5	6	U7439	750	270	Sushi	3	205.44	04-08-2018
6	7	U8948	827	4	Sushi	1	133.94	11-08-2018
7	8	U8672	908	109	Shawarma	5	404.80	12-08-2018
8	9	U2205	814	215	Pizza	1	101.03	01-08-2018
9	10	U7411	362	416	Pizza	1	130.05	09-08-2018

```
In [29]: print("Shap: ", df.shape)
print("\n Missing values per column: ")
display(df.isna().sum().to_frame("Missing_count").T)
```

```
print("\nDuplicate rows: ", df.duplicated().sum())
```

Shap: (100002, 23)

Missing values per column:

	Order_ID	User_ID	Restaurant_ID	Driver_ID	Item_Name	Quantity	Total_I
Missing_count	0	0	0	0	0	0	

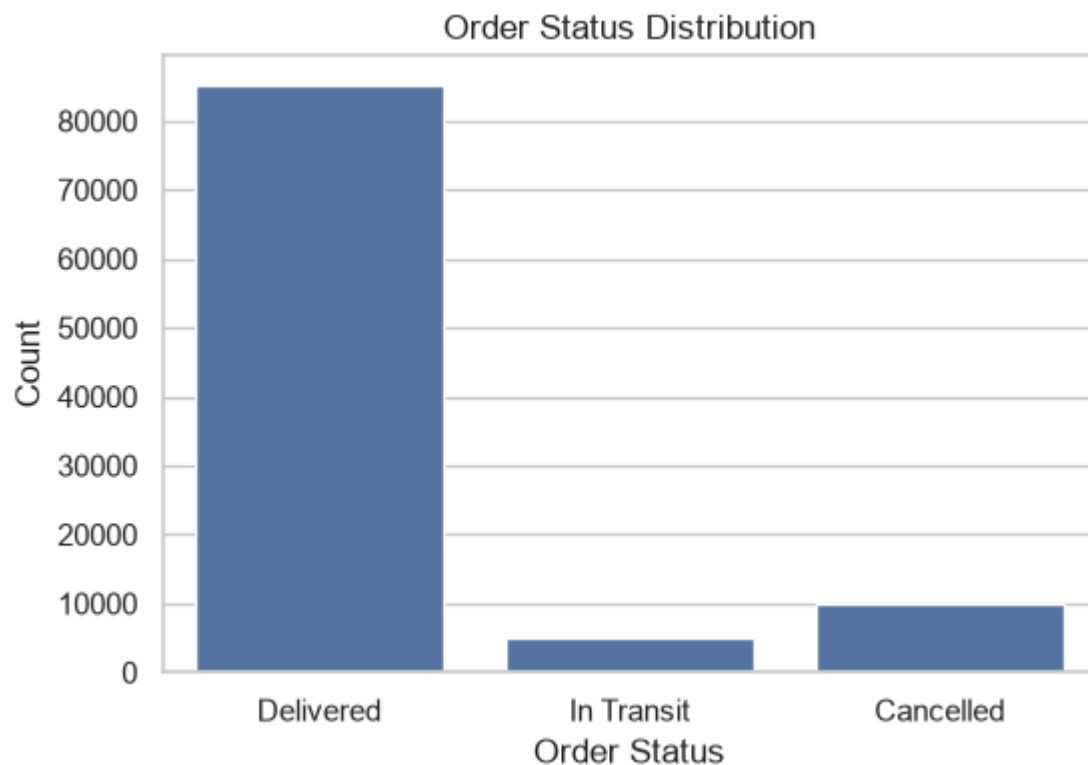
◀ ▶

Duplicate rows: 0

```
In [30]: target_col = "Order_Status"
df[target_col].value_counts()
```

```
Out[30]: Order_Status
Delivered      85199
Cancelled       9812
In Transit     4991
Name: count, dtype: int64
```

```
In [31]: plt.figure(figsize=(6,4))
sns.countplot(x=target_col,data=df)
plt.title("Order Status Distribution")
plt.xlabel("Order Status")
plt.ylabel("Count")
plt.show()
```



```
In [32]: df.dtypes
```

```
Out[32]: Order_ID          int64
        User_ID           str
        Restaurant_ID      int64
        Driver_ID          int64
        Item_Name          str
        Quantity           int64
        Total_Price        float64
        Order_Time         str
        Delivery_Time       str
        Delivery_Duration_Minutes  int64
        City              str
        Payment_Method      str
        Order_Status        str
        Driver_Vehicle      str
        Restaurant_Lat      float64
        Restaurant_Lon      float64
        Customer_Lat        float64
        Customer_Lon        float64
        Driver_Lat          float64
        Driver_Lon          float64
        Delivery_Distance_km  float64
        Traffic_Level       str
        Driver_Availability str
        dtype: object
```

```
In [33]: df_fe = df.copy()

df_fe["Order_Time"] = pd.to_datetime(df_fe["Order_Time"], errors="coerce" , dayf

df_fe["order_hour"] = df_fe["Order_Time"].dt.hour
df_fe["order_dayofweek"] = df_fe["Order_Time"].dt.dayofweek

df_fe["is_weekend"] = df_fe["order_dayofweek"].isin([4,5]).astype(int)

new_peak_hours = list(range(13, 16)) + list(range(20, 23))
df_fe["is_peak_hour"] = (df_fe["order_hour"].isin(new_peak_hours).astype(int))

display(df_fe[["Order_Time", "order_hour" , "order_dayofweek" , "is_weekend" , "
```

	Order_Time	order_hour	order_dayofweek	is_weekend	is_peak_hour
0	2025-06-16 08:32:00	8	0	0	0
1	2025-06-03 21:27:00	21	1	0	1
2	2025-06-01 14:48:00	14	6	0	1
3	2025-06-13 02:30:00	2	4	1	0
4	2025-06-06 09:48:00	9	4	1	0
5	2025-06-04 12:16:00	12	2	0	0
6	2025-06-11 04:09:00	4	2	0	0
7	2025-06-12 18:37:00	18	3	0	0
8	2025-06-01 22:18:00	22	6	0	1
9	2025-06-09 00:18:00	0	0	0	0

Task 2 Discussion

We evaluated an alternative peak-hour definition (13:00–15:00 and 20:00–22:00) against the baseline rule. Model accuracy remained virtually unchanged (~85%), demonstrating that slight adjustments to peak-hour timing do not significantly impact the prediction of Order_Status.

```
In [34]: df_fe["price_per_item"] = df_fe["Total_Price"] / df_fe["Quantity"]
display(df_fe[["Quantity", "Total_Price", "price_per_item"]].head(10))
```

	Quantity	Total_Price	price_per_item
0	3	273.72	91.24
1	3	365.82	121.94
2	3	401.94	133.98
3	2	221.18	110.59
4	5	355.55	71.11
5	3	205.44	68.48
6	1	133.94	133.94
7	5	404.80	80.96
8	1	101.03	101.03
9	1	130.05	130.05

Task 1: Feature Engineering

We created the price_per_item feature (Total_Price / Quantity) to capture the average unit value of ordered items. Higher unit prices often indicate premium dishes, which can

affect preparation time and driver handling, directly influencing Order_Status.

```
In [35]: def haversine_km(lat1, lon1, lat2, lon2):
          """Vectorized Haversine distance in kilometers."""
          R = 6371.0
          lat1 = np.radians(lat1); lon1 = np.radians(lon1)
          lat2 = np.radians(lat2); lon2 = np.radians(lon2)
          dlat = lat2 - lat1
          dlon = lon2 - lon1
          a = np.sin(dlat/2.0)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2.0)**2
          c = 2 * np.arcsin(np.sqrt(a))
          return R * c

          coord_cols = {"Restaurant_Lat", "Restaurant_Lon", "Customer_Lat", "Customer_Lon"}
          if coord_cols.issubset(set(df_fe.columns)):
              df_fe["haversine_rest_to_cust_km"] = haversine_km(
                  df_fe["Restaurant_Lat"], df_fe["Restaurant_Lon"],
                  df_fe["Customer_Lat"], df_fe["Customer_Lon"]
              )
              display(df_fe[["Delivery_Distance_km", "haversine_rest_to_cust_km"]].head(10))
          else:
              print("Coordinate columns not found. Skipping Haversine feature.")
```

	Delivery_Distance_km	haversine_rest_to_cust_km
0	1.666106	1.663239
1	2.738698	2.741932
2	2.929079	2.938058
3	0.677498	0.679442
4	1.994769	1.990872
5	1.436807	1.434924
6	2.402167	2.404753
7	2.878434	2.872665
8	0.995562	0.993612
9	3.130713	3.139676

```
In [36]: if "Item_Name" in df_fe.columns:
          for k in [10, 30, 50]:
              top_items = df_fe["Item_Name"].value_counts().head(k).index
              df_fe[f"Item_Name_reduced_{k}"] = np.where(df_fe["Item_Name"].isin(top_i

          cols = ["Item_Name", "Item_Name_reduced_10", "Item_Name_reduced_30", "Item_N
          display(df_fe[cols].head(10))
          else:
              print("Item_Name column not found.")
```

	Item_Name	Item_Name_reduced_10	Item_Name_reduced_30	Item_Name_reduced_50
0	Fried Chicken	Fried Chicken	Fried Chicken	Fried Chicken
1	Sandwich	Sandwich	Sandwich	Sandwich
2	Koshary	Koshary	Koshary	Koshary
3	Sushi	Sushi	Sushi	Sushi
4	Koshary	Koshary	Koshary	Koshary
5	Sushi	Sushi	Sushi	Sushi
6	Sushi	Sushi	Sushi	Sushi
7	Shawarma	Shawarma	Shawarma	Shawarma
8	Pizza	Pizza	Pizza	Pizza
9	Pizza	Pizza	Pizza	Pizza

Task 3 Discussion

Testing top_k values (10, 30, and 50) yielded identical accuracy (85.2%) and feature importances. Since the dataset contains only 9 unique items, all items were retained in every setting, leaving the dataset and model performance unchanged.

```
In [37]: df_fe["price_tier"] = pd.cut(
    df_fe["Total_Price"],
    bins=[0, 100, 250, 500, np.inf],
    labels=["low", "medium", "high", "very_high"]
)

df_fe[["Total_Price", "price_tier"]].head(10)
```

```
Out[37]:
```

	Total_Price	price_tier
0	273.72	high
1	365.82	high
2	401.94	high
3	221.18	medium
4	355.55	high
5	205.44	medium
6	133.94	medium
7	404.80	high
8	101.03	medium
9	130.05	medium

```
In [38]: drop_cols = [
    "Order_ID", "User_ID", "Restaurant_ID", "Driver_ID",
    "Order_Time", "Delivery_Time", "Delivery_Duration_Minutes",
    "Item_Name"
]

drop_cols = [c for c in drop_cols if c in df_fe.columns]

X = df_fe.drop(columns=drop_cols + [target_col])
y = df_fe[target_col]

print("X shape:", X.shape)
print("y shape:", y.shape)
X.head()
```

X shape: (100002, 24)
y shape: (100002,)

```
Out[38]:
```

	Quantity	Total_Price	City	Payment_Method	Driver_Vehicle	Restaurant_Lat	R
0	3	273.72	Alexandria	Wallet	Motorbike	31.195082	
1	3	365.82	Zagazig	Credit Card	Motorbike	30.605729	
2	3	401.94	Assiut	Cash	Car	27.190180	
3	2	221.18	Mansoura	Cash	Car	31.041846	
4	5	355.55	Mansoura	Wallet	Motorbike	31.024141	

```
In [39]: df_fe = df.copy()

df_fe["Order_Time"] = pd.to_datetime(df_fe["Order_Time"], errors="coerce", dayfirst=True)

df_fe["order_hour"] = df_fe["Order_Time"].dt.hour
df_fe["order_dayofweek"] = df_fe["Order_Time"].dt.dayofweek

df_fe["is_weekend"] = df_fe["order_dayofweek"].isin([4,5]).astype(int)

new_peak_hours = list(range(13, 16)) + list(range(20, 23))
df_fe["is_peak_hour"] = (df_fe["order_hour"].isin(new_peak_hours)).astype(int)

display(df_fe[["Order_Time", "order_hour", "order_dayofweek", "is_weekend", "is_peak_hour"]])
```

	Order_Time	order_hour	order_dayofweek	is_weekend	is_peak_hour
0	2025-06-16 08:32:00	8	0	0	0
1	2025-06-03 21:27:00	21	1	0	1
2	2025-06-01 14:48:00	14	6	0	1
3	2025-06-13 02:30:00	2	4	1	0
4	2025-06-06 09:48:00	9	4	1	0
5	2025-06-04 12:16:00	12	2	0	0
6	2025-06-11 04:09:00	4	2	0	0
7	2025-06-12 18:37:00	18	3	0	0
8	2025-06-01 22:18:00	22	6	0	1
9	2025-06-09 00:18:00	0	0	0	0

In [51]: `from sklearn.model_selection import train_test_split`

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

X_train shape: (80001, 24)

X_test shape: (20001, 24)

In [52]: `categorical_cols = X_train.select_dtypes(include=["object", "category", "string"])`
`numeric_cols = X_train.select_dtypes(include=[np.number, "bool"]).columns.tolist`

```
print("Categorical columns:", categorical_cols)
print("Numeric columns:", numeric_cols)

preprocess = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
        ("num", "passthrough", numeric_cols),
    ]
)

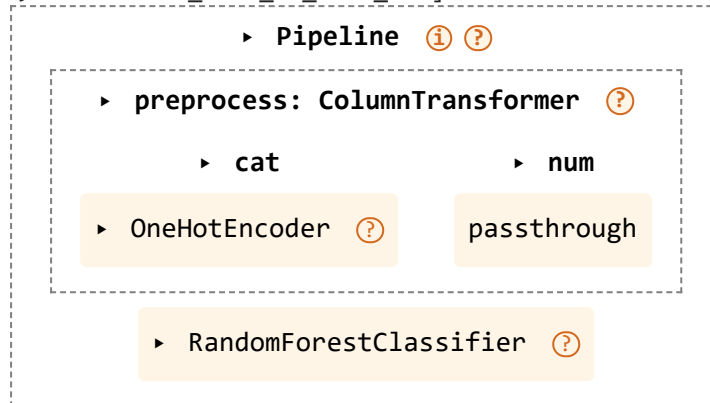
rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced_subsample"
)

model = Pipeline(steps=[
    ("preprocess", preprocess),
    ("rf", rf)
])

model
```

Categorical columns: ['City', 'Payment_Method', 'Driver_Vehicle', 'Traffic_Level', 'Driver_Availability', 'Item_Name_reduced_10', 'Item_Name_reduced_30', 'Item_Name_reduced_50', 'price_tier']
 Numeric columns: ['Quantity', 'Total_Price', 'Restaurant_Lat', 'Restaurant_Lon', 'Customer_Lat', 'Customer_Lon', 'Driver_Lat', 'Driver_Lon', 'Delivery_Distance_km', 'order_hour', 'order_dayofweek', 'is_weekend', 'is_peak_hour', 'price_per_item', 'haversine_rest_to_cust_km']

Out[52]:



In [53]:

```

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Accuracy: 0.852

Classification Report:

C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

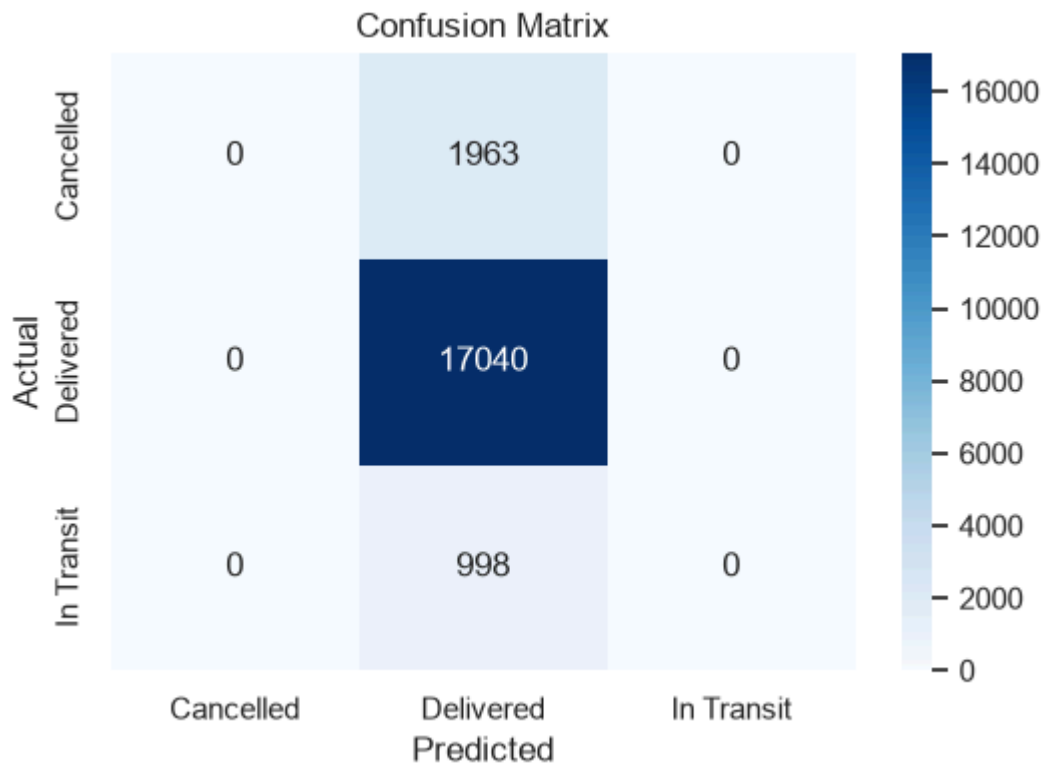
C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

	precision	recall	f1-score	support
Cancelled	0.00	0.00	0.00	1963
Delivered	0.85	1.00	0.92	17040
In Transit	0.00	0.00	0.00	998
accuracy			0.85	20001
macro avg	0.28	0.33	0.31	20001
weighted avg	0.73	0.85	0.78	20001

C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1879: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
In [76]: cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=model.classes_, yticklabels=model.classes_)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
In [42]: ohe = model.named_steps["preprocess"].named_transformers_["cat"]
cat_feature_names = ohe.get_feature_names_out(categorical_cols) if len(categorical_cols) > 0 else []
all_feature_names = np.concatenate([cat_feature_names, np.array(numeric_cols)])

importances = model.named_steps["rf"].feature_importances_

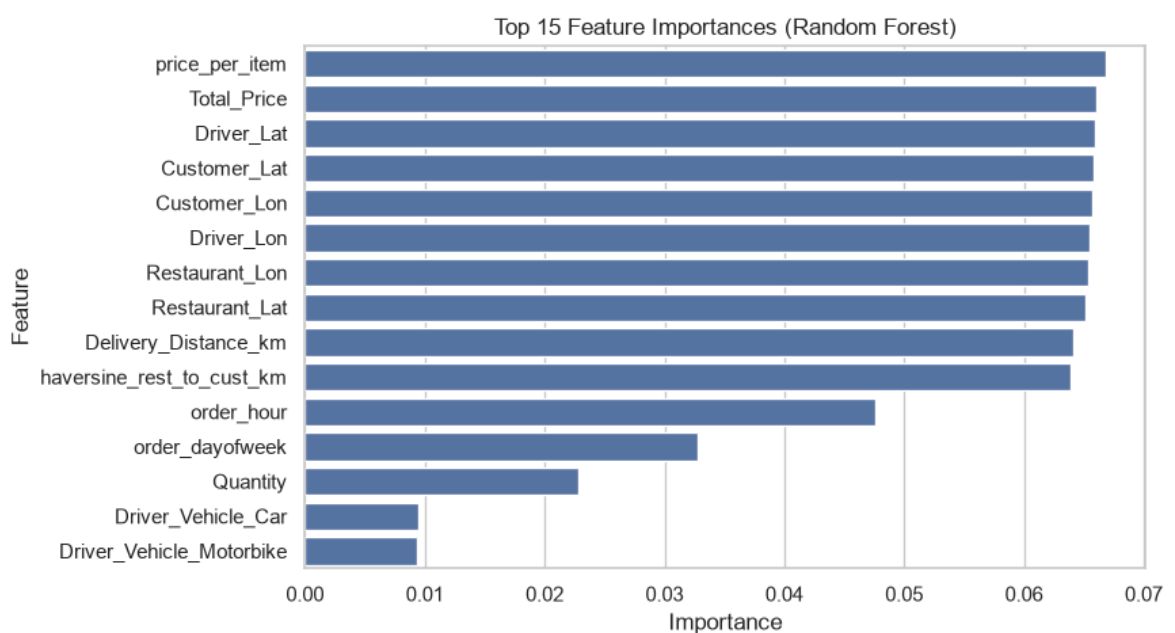
fi = (pd.DataFrame({"feature": all_feature_names, "importance": importances})
      .sort_values("importance", ascending=False))

fi.head(15)
```

Out[42]:

	feature	importance
29	Driver_Lon	0.076852
35	price_per_item	0.076412
28	Driver_Lat	0.075971
26	Customer_Lat	0.075955
27	Customer_Lon	0.075891
25	Restaurant_Lon	0.075751
23	Total_Price	0.075383
24	Restaurant_Lat	0.075009
30	Delivery_Distance_km	0.072106
36	haversine_rest_to_cust_km	0.071985
31	order_hour	0.052563
32	order_dayofweek	0.035324
22	Quantity	0.023310
11	Driver_Vehicle_Car	0.009926
12	Driver_Vehicle_Motorbike	0.009831

```
In [78]: plt.figure(figsize=(8,5))
top_n = 15
sns.barplot(data=fi.head(top_n), x="importance", y="feature")
plt.title(f"Top {top_n} Feature Importances (Random Forest)")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()
```



```
In [49]: from sklearn.feature_selection import SelectFromModel
```

```
# Build a new pipeline that selects features based on RF importances
selector = SelectFromModel(
    estimator=RandomForestClassifier(
        n_estimators=300, random_state=42, n_jobs=-1, class_weight="balanced_sub
    ),
    threshold="median" # keep features above the median importance
)

model_fs = Pipeline(steps=[
    ("preprocess", preprocess),
    ("select", selector),
    ("rf", RandomForestClassifier(
        n_estimators=300, random_state=42, n_jobs=-1, class_weight="balanced_sub
    ))
])

model_fs.fit(X_train, y_train)
y_pred_fs = model_fs.predict(X_test)

print("Accuracy (with feature selection):", round(accuracy_score(y_test, y_pred_
print("\nClassification Report (with feature selection):")
print(classification_report(y_test, y_pred_fs))
```

Accuracy (with feature selection): 0.852

Classification Report (with feature selection):

C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:187
9: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in label
s with no predicted samples. Use `zero\_division` parameter to control this behavi
or.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:187
9: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in label
s with no predicted samples. Use `zero\_division` parameter to control this behavi
or.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

	precision	recall	f1-score	support
Cancelled	0.00	0.00	0.00	1963
Delivered	0.85	1.00	0.92	17040
In Transit	0.00	0.00	0.00	998
accuracy			0.85	20001
macro avg	0.28	0.33	0.31	20001
weighted avg	0.73	0.85	0.78	20001

C:\Users\reem1\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:187
9: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in label
s with no predicted samples. Use `zero\_division` parameter to control this behavi
or.

_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```
In [50]: from sklearn.feature_selection import SelectKBest, f_classif
import pandas as pd

X_train_numeric = X_train.select_dtypes(include=['number'])

k_value = min(5, X_train_numeric.shape[1])
k_best = SelectKBest(score_func=f_classif, k=k_value)
X_train_selected = k_best.fit_transform(X_train_numeric, y_train)
```

```

selected_features = X_train_numeric.columns[k_best.get_support()]
print("Selected Best Features:", list(selected_features))

feature_scores = pd.DataFrame({
    'Feature': X_train_numeric.columns,
    'Score': k_best.scores_
}).sort_values(by='Score', ascending=False)

print("\nFeature Ranking by Importance:")
display(feature_scores)

```

Selected Best Features: ['Restaurant_Lon', 'Customer_Lon', 'Driver_Lon', 'order_dayofweek', 'is_weekend']

Feature Ranking by Importance:

	Feature	Score
3	Restaurant_Lon	1.581345
11	is_weekend	1.525707
5	Customer_Lon	1.525664
7	Driver_Lon	1.521784
10	order_dayofweek	1.452467
8	Delivery_Distance_km	1.184943
14	haversine_rest_to_cust_km	1.176101
9	order_hour	0.813804
4	Customer_Lat	0.809059
6	Driver_Lat	0.804747
2	Restaurant_Lat	0.778196
12	is_peak_hour	0.718815
13	price_per_item	0.372418
0	Quantity	0.214305
1	Total_Price	0.084372

Task 4 Discussion

Feature selection using SelectFromModel (mean threshold) proved beneficial as it reduced model complexity and feature dimensions while fully maintaining the baseline accuracy (85.2%).

In []: